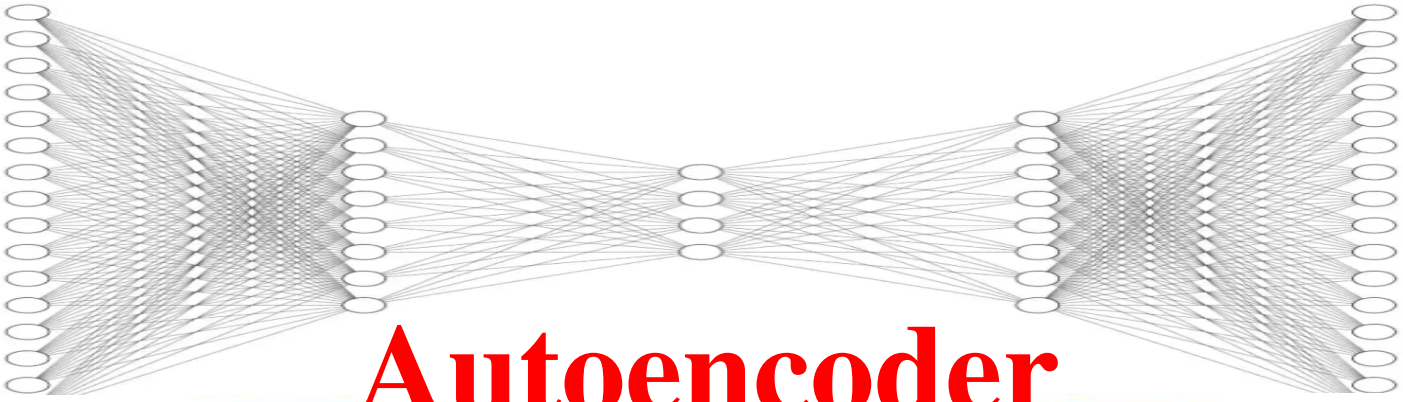
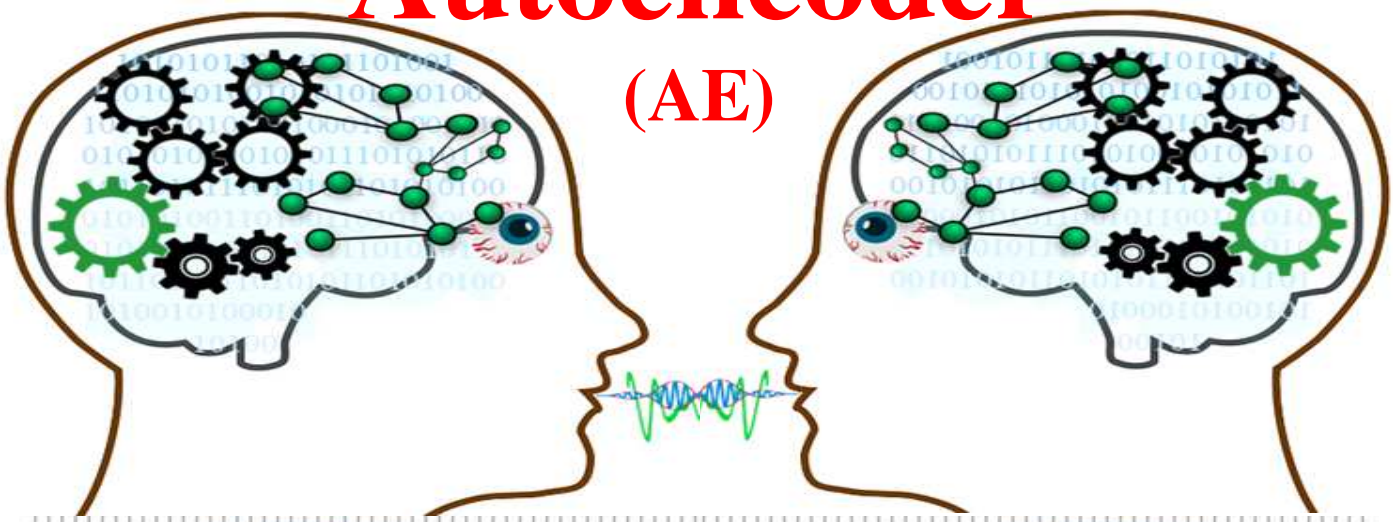
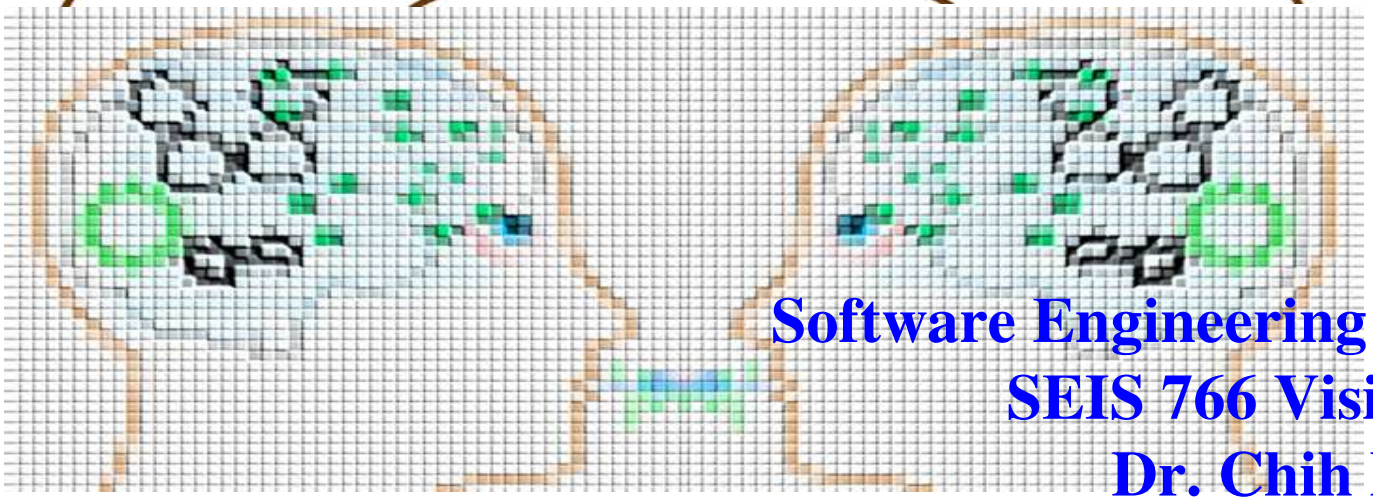


Autoencoder

(AE)





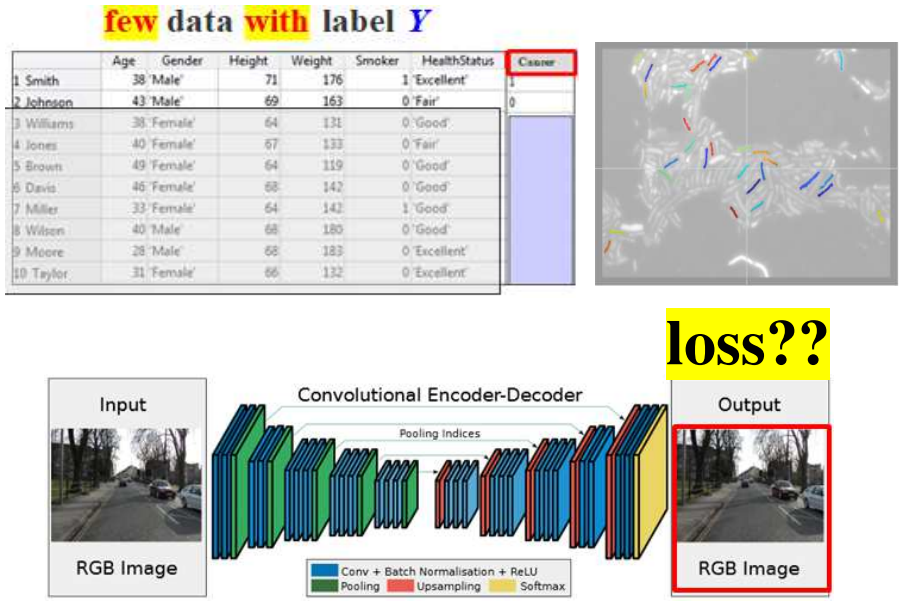
Software Engineering & Data Science

SEIS 766 Vision A.I.

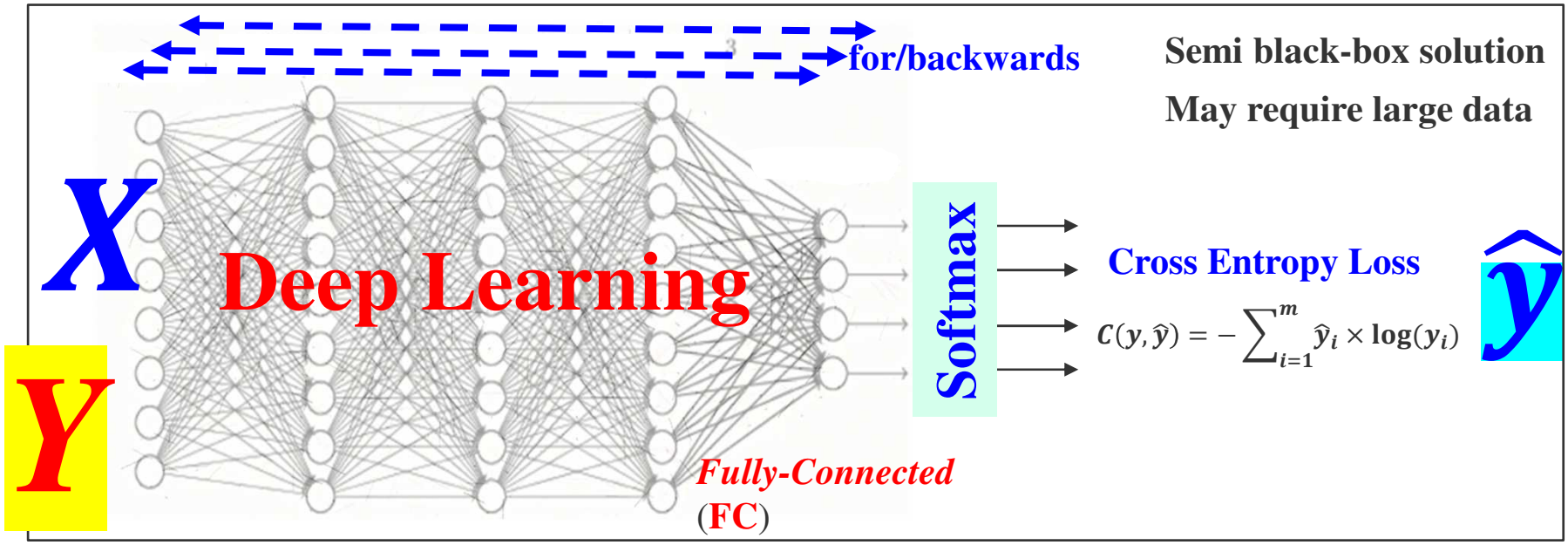
Dr. Chih Lai

Outline

- *Semi*-Supervised Learning with Autoencoder (AE)
- Training / Fine-Tuning Autoencoder / **Programming**
  - **Not Enough *Ys***, Dimension Reduction ***Z***, Train Deep NN w/ Few ***Y***s, More……
- Homework 4 Review
- AE with Convolution, Denoising Autoencoder
- ***Variational Autoencoder***



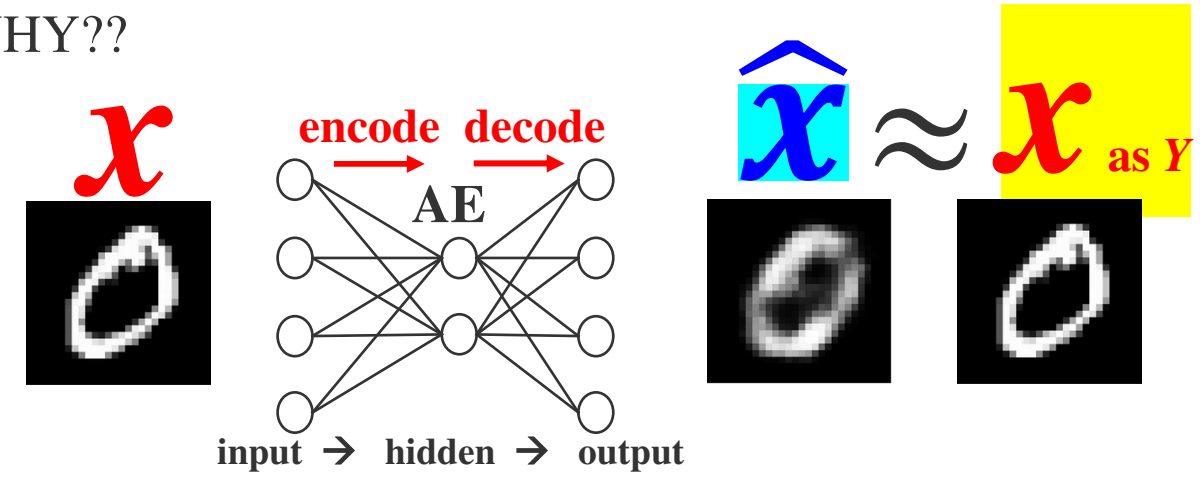
Training in Regular NN vs. Autoencoder

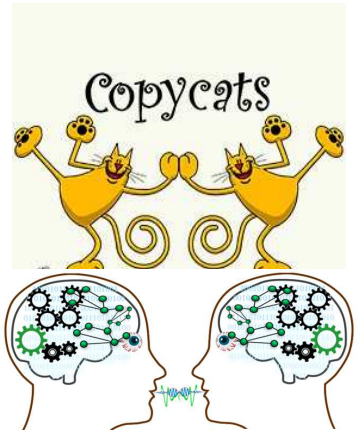


- Training autoencoder (**AE**), **WHAT IS** ***Y***? ➔ **semi-supervised** learning

SSL (self-supe
Unsupervised
Distillation

- But, WHY??





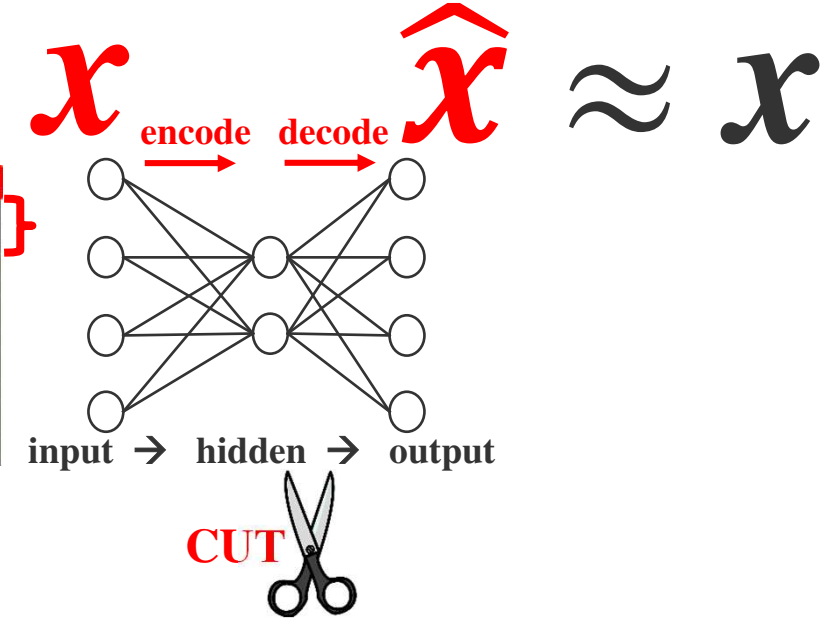
App 1. Classifying Majority of Unlabeled Data (or few available *Y*s)

- Autoencoder ➔ **semi-supervised learning…** NO need to have (a lot of) *Y*

Training AE

Not enough *Y*

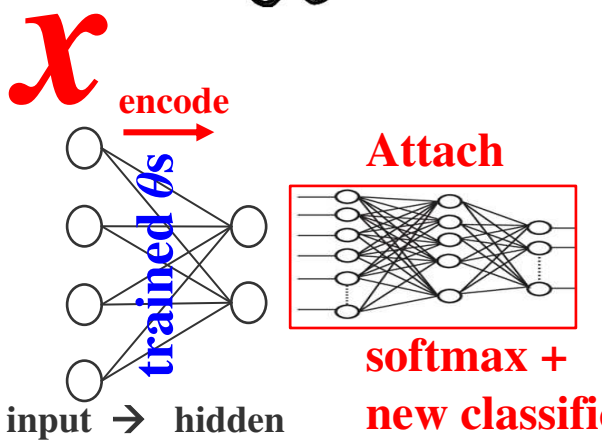
	Age	Gender	Height	Weight	Smoker	HealthStatus	Cancer
1 Smith	38	'Male'	71	176	1	'Excellent'	1
2 Johnson	43	'Male'	69	163	0	'Fair'	0
3 Williams	38	'Female'	64	131	0	'Good'	
4 Jones	40	'Female'	67	133	0	'Fair'	
5 Brown	49	'Female'	64	119	0	'Good'	
6 Davis	46	'Female'	68	142	0	'Good'	
7 Miller	33	'Female'	64	142	1	'Good'	
8 Wilson	40	'Male'	68	180	0	'Good'	
9 Moore	28	'Male'	68	183	0	'Excellent'	
10 Taylor	31	'Female'	66	132	0	'Excellent'	



***Fine-tuning* training using**

few data with label *Y*

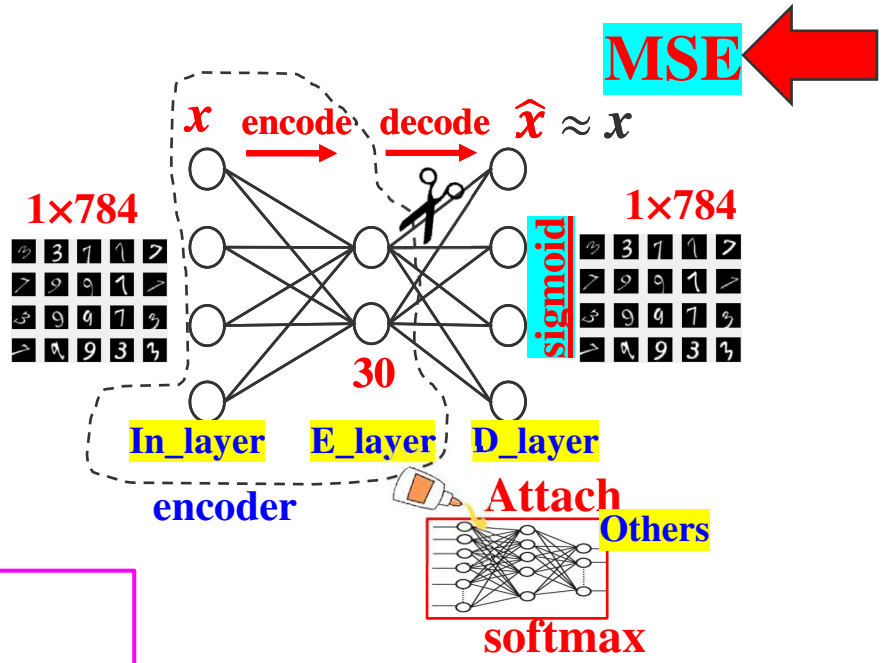
	Age	Gender	Height	Weight	Smoker	HealthStatus	Cancer
1 Smith	38	'Male'	71	176	1	'Excellent'	1
2 Johnson	43	'Male'	69	163	0	'Fair'	0
3 Williams	38	'Female'	64	131	0	'Good'	
4 Jones	40	'Female'	67	133	0	'Fair'	
5 Brown	49	'Female'	64	119	0	'Good'	
6 Davis	46	'Female'	68	142	0	'Good'	
7 Miller	33	'Female'	64	142	1	'Good'	
8 Wilson	40	'Male'	68	180	0	'Good'	
9 Moore	28	'Male'	68	183	0	'Excellent'	
10 Taylor	31	'Female'	66	132	0	'Excellent'	



We will make this VERY deep later!!!

Keras Autoencoder Functional API

- Build few AEs of **1 *layer***
  - https://wiseodd.github.io/techblog/2016/12/03/autoencoders/
  - https://blog.keras.io/building-autoencoders-in-keras.html



VAI_Slide_AE_1_layer.ipyng
VAI_Slide_2AEs_Soft_then_Stack.ipynb
VAI_Slide_2AEs_then_Stack_Soft.ipynb

```
original_dim = 784
In_layer = Input(shape=(original_dim,))
# encodede layer
encoding_dim = 30
E_layer = Dense(encoding_dim, activation='relu')(In_layer)
# decode layer
D_layer = Dense(original_dim, activation='sigmoid')(E_layer)

# put together
autoencoder = Model(In_layer, D_layer)

# loss='mean_squared_error'
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
autoencoder.fit(x_train, x_train, epochs=50, batch_size=256,
                validation_data=(x_test, x_test))
```

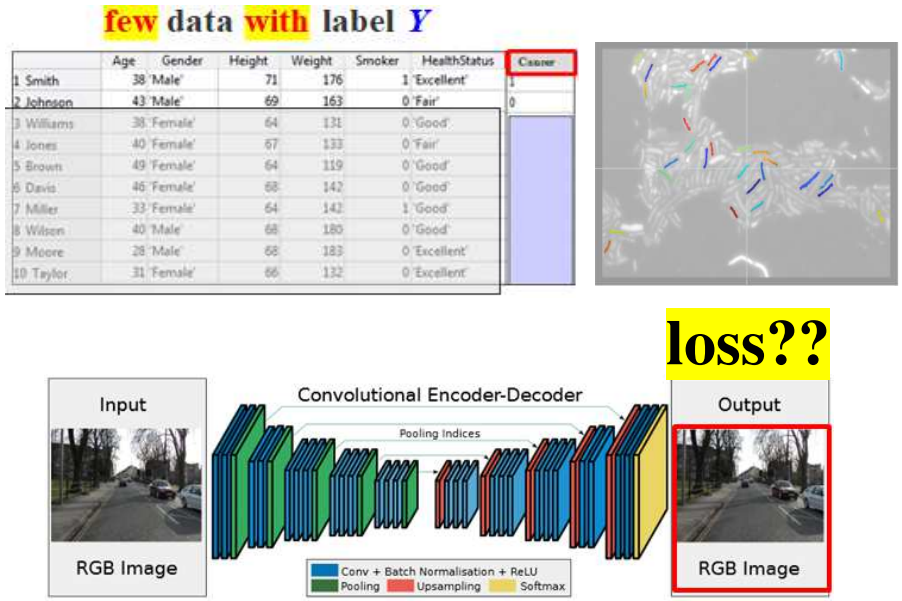
```
### create others
Other = Dense(#classes, ...
                activation='softmax')(E_layer)
# attach together
Final_NN = Model(In_layer, Other)
Final_NN.fit(x_train, Y_train, ...)

-------------------------------------------

### Get Encoder ONLY for Z
encoder = Model(In_layer, E_layer)
Z = encoder.predict(X)      # Z = 30-D
```

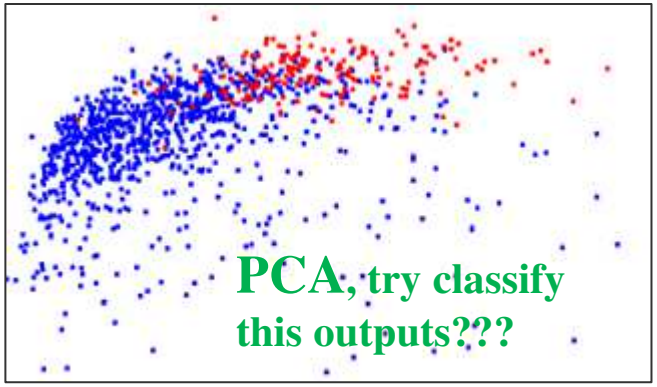
Outline

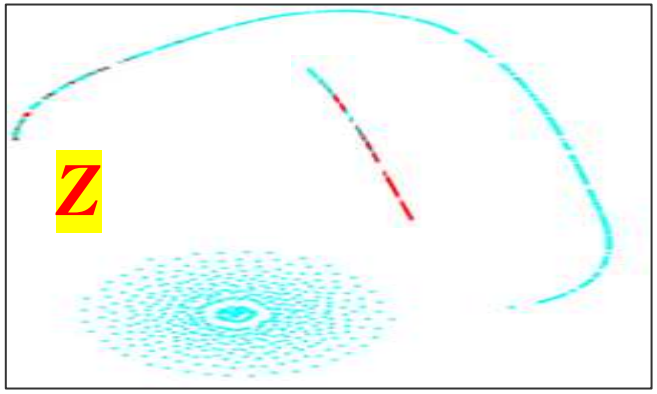
- *Semi*-Supervised Learning with Autoencoder (AE)
- Training / Fine-Tuning Autoencoder / Programming
  - Not Enough *Ys*, **Dimension Reduction *Z*, Train Deep NN w/ Few *Ys***, More……
- Homework 4 Review
- AE with Convolution, Denoising Autoencoder
- ***Variational Autoencoder***



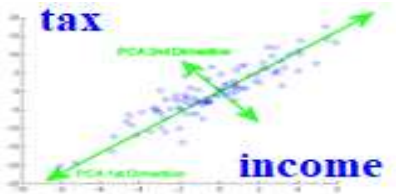
App 2. Dimensionality Reduction

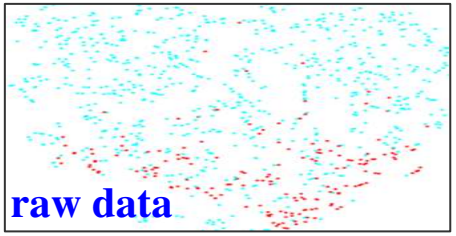
- Will ***Z***-Code offer better classification quality than PCA???





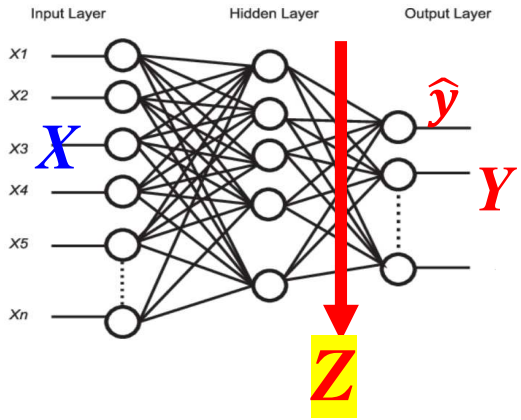
	Age	Gender	Height	Weight	Smoker	HealthStatus	Cluster
1 Smith	38	'Male'	71	176	1	'Excellent'	
2 Johnson	43	'Male'	69	163	0	'Fair'	
3 Williams	38	'Female'	64	131	0	'Good'	
4 Jones	40	'Female'	67	133	0	'Fair'	
5 Brown	49	'Female'	64	119	0	'Good'	
6 Davis	46	'Female'	68	142	0	'Good'	
7 Miller	33	'Female'	64	142	1	'Good'	
8 Wilson	40	'Male'	68	180	0	'Good'	
9 Moore	28	'Male'	68	183	0	'Excellent'	
10 Taylor	31	'Female'	66	132	0	'Excellent'	

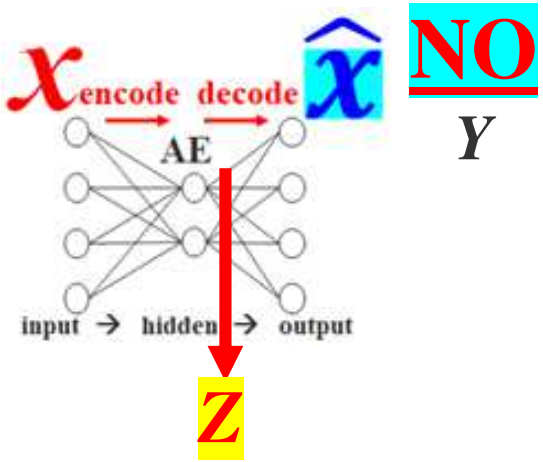




- But, how to get ***Z***-code **w/o** ***Y***? ➔ **AE** = semi-supervised learning

Get ***Z*** using ***X*** **AND** ***Y***





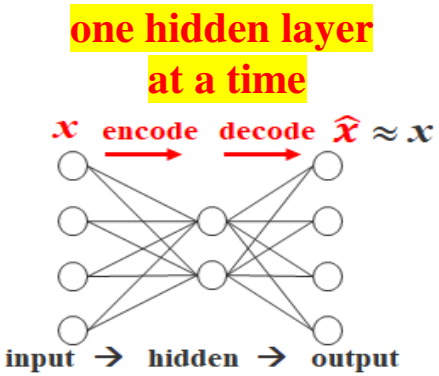
NO

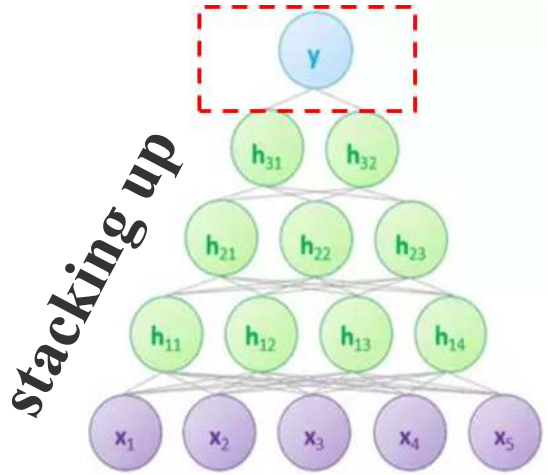
Y

App 3:
speed up
training of VERY
deep NN… HOW??

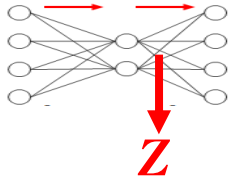
Just Brute-and-Force Training Deep NN w/ Few *Y*s?? App 3

- Train **deep** NN w/ many layers can be **difficult** in practice **w/ only few labeled data**
- Another way to effectively train a **deep** NN is: training **one layer at a time**
  - Training **1 layer at a time** using **autoencoder**, then **stack up** layers
- Detail steps (example in next slides):
  1. First train **one hidden layer at a time** in a **semi-supervised** fashion w/ autoencoder
  2. Then you train a final softmax layer in ***supervised*** fashion
  3. **Stack** all the "***encoders***" together to form a **deep** NN
  4. **Fine-tune** training the **deep** NN one last time in a ***supervised*** fashion

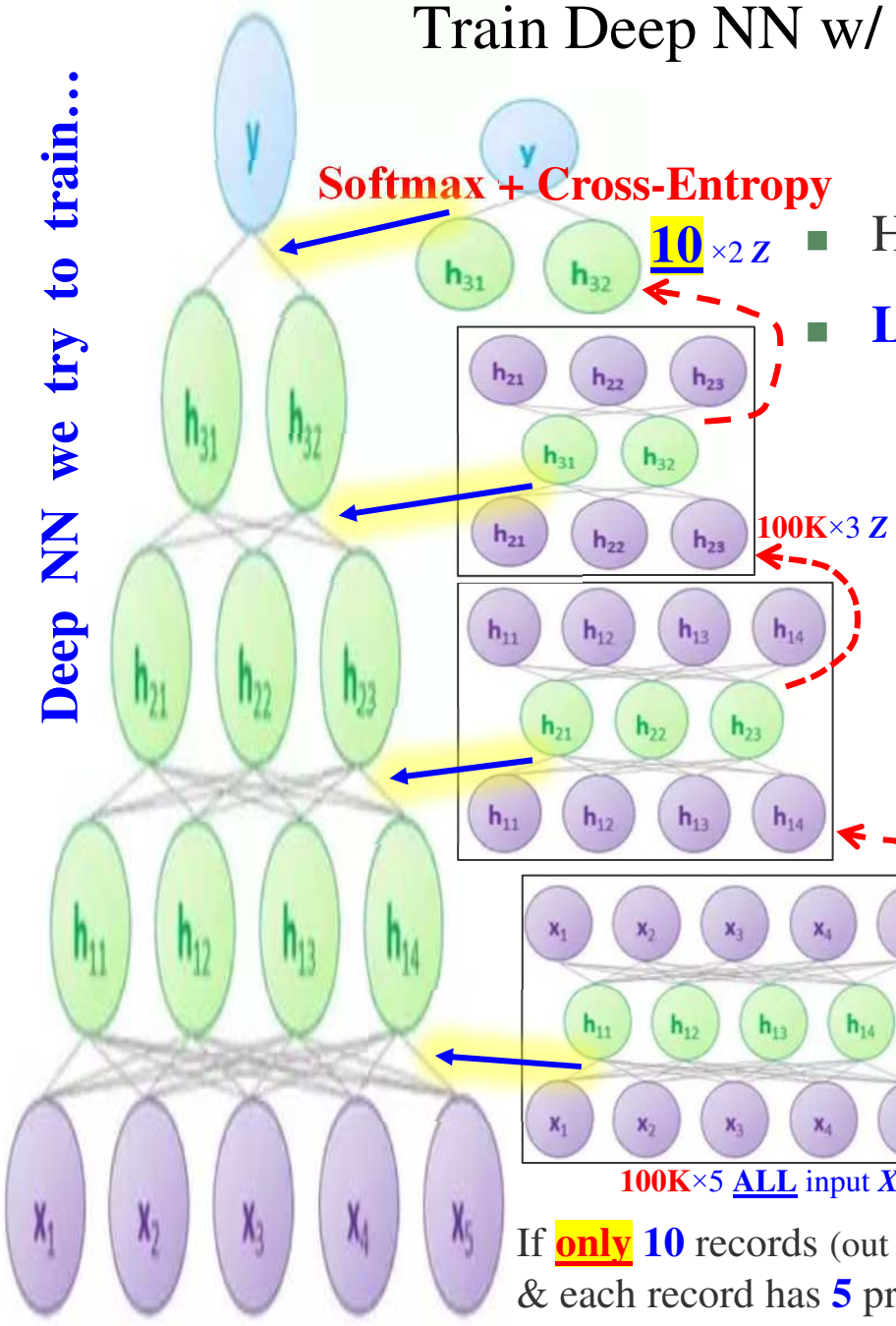




Train Deep NN w/ AE, One Layer A Time



Deep NN we try to train…



- How do we initialize θs in **deep** NN?
- **Layer-wise initialize & pre-train** an AE
  - Use **ALL unlabeled** data for training **AE**
  - Get **Z**s from encoder to train next **AE**
  - **…… repeat……..**
  - Stack **weights θs** from all **encoders** together
  - Finally, use small amount of **labeled** data for **fine-tuning** the deep NN

WHAT is the loss function for each training??

If **only 10** records (out of **100K**) have ***Y***s & each record has **5** predictors

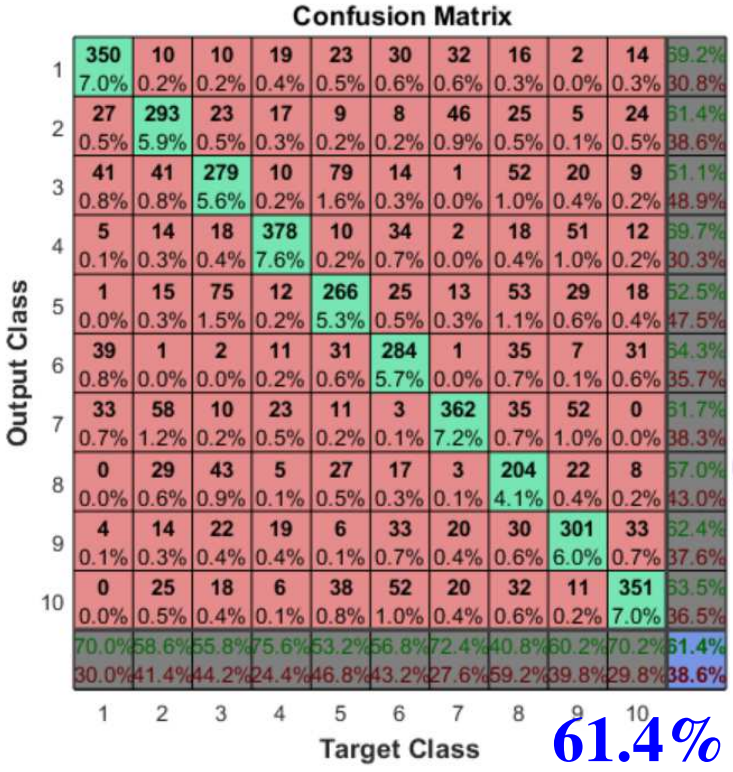
 https://www.quora.com/How-is-deep-learning-different-from-multilayer-perceptron#

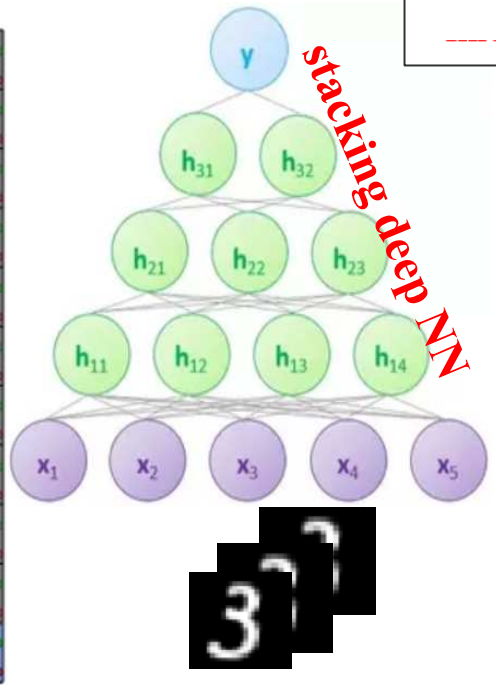
Disappointing Result after **Stacking** together?

- Why bother doing this for 61.4% accuracy???
  - But, it is already better than

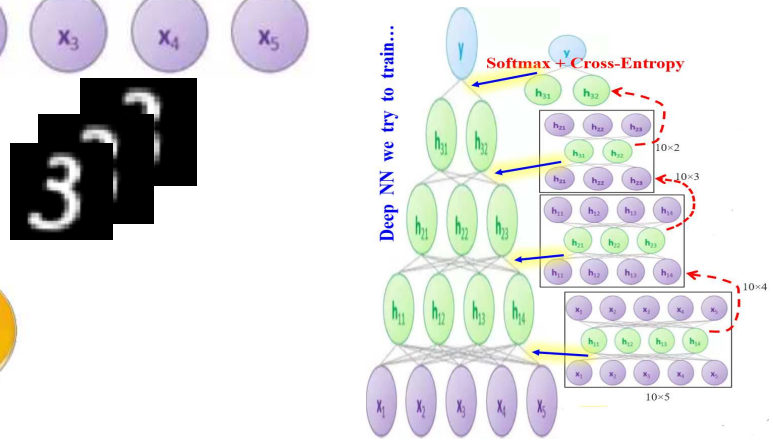
    direct training deep NN w/ few (i.e. 10) *Y*s
  - In fact, it can be **EVEN BETTER**!!! **HOW??**

- Use all **unlabeled** data for training **AE**
- Get ***Z***s from previous encoder to train next **AE**
- ..... **repeat**.....
- Stack weights θs from all **encoder**s together



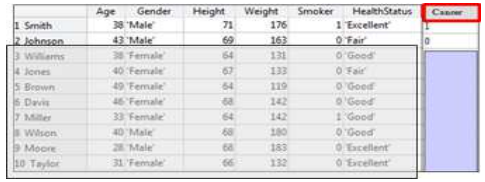


	Age	Gender	Height	Weight	Smoker	HealthStatus	Cancer
1 Smith	38	'Male'	71	176	1	'Excellent'	1
2 Johnson	43	'Male'	69	163	0	'Fair'	0
3 Williams	38	'Female'	64	131	0	'Good'	
4 Jones	40	'Female'	67	133	0	'Fair'	
5 Brown	49	'Female'	64	119	0	'Good'	
6 Davis	46	'Female'	68	142	0	'Good'	
7 Miller	33	'Female'	64	142	1	'Good'	
8 Wilson	40	'Male'	68	180	0	'Good'	
9 Moore	28	'Male'	66	183	0	'Excellent'	
10 Taylor	31	'Female'	66	132	0	'Excellent'	



61.4% disappointed??

One More Step➔ **Fine-Tuning** Stacked DNN

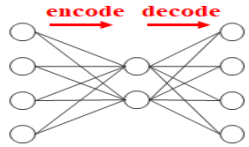


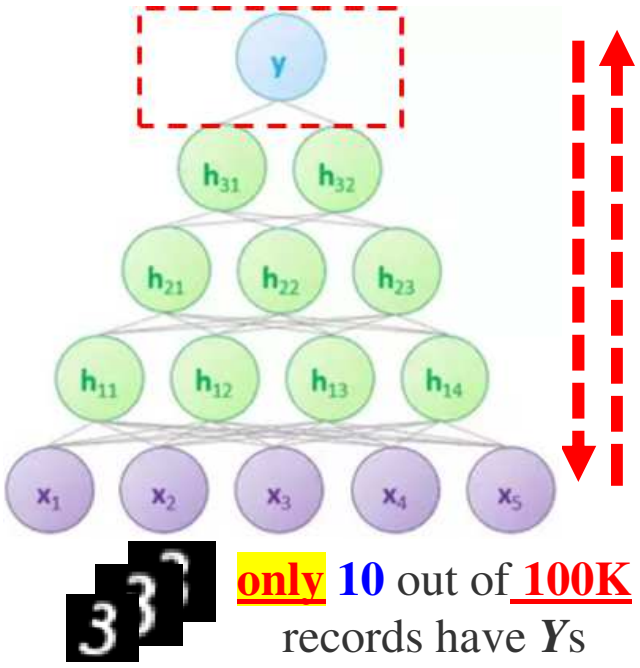
- Feed **few** **training** data w/ ***Y*** into **stacked** DNN again for ***fine-tuning*** (**training**) θs
  - Accuracy improves to **98.4%**

- AE **alone** is **NOT** a classifier… i.e. AE ≠ classifiers…

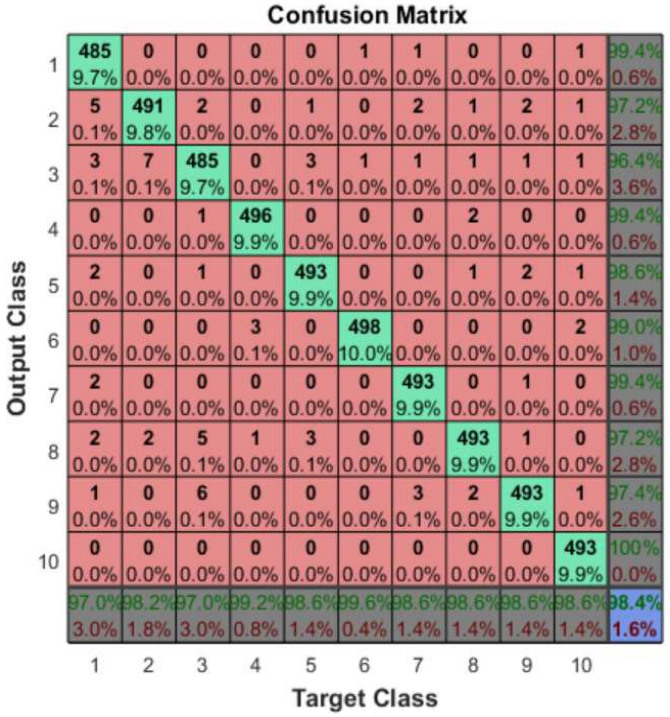
Programming???

  - Pretrain & stack multiple ***AE***s into **ONE** final deep **classifier** that is potentially **easier to train & produce better quality**





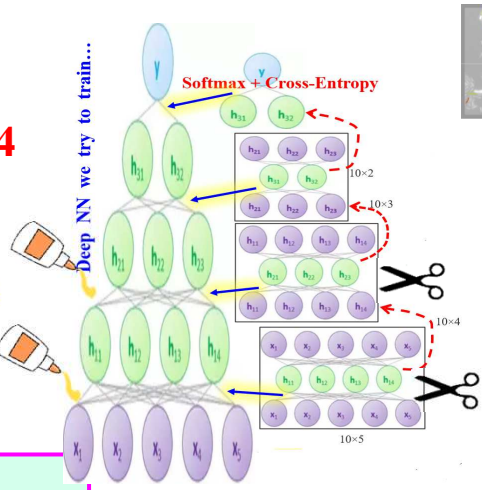
only **10** out of **100K** records have ***Y***s

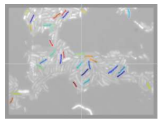


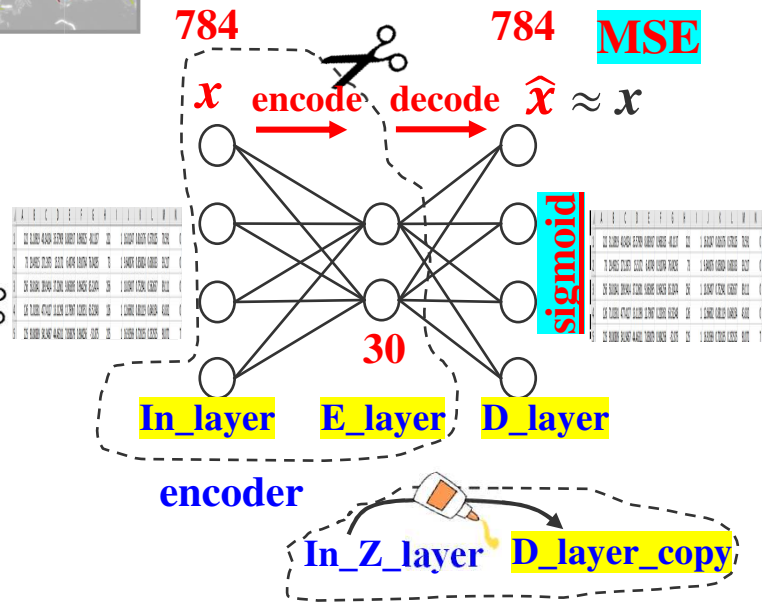
98.4%

Keras Autoencoder Functional API for HW#4

- Build few AEs of **1 *layer***
  - https://wiseodd.github.io/techblog/2016/12/03/autoencoders/
  - https://blog.keras.io/building-autoencoders-in-keras.html







VAI_Slide_AE_1_layer.ipyng
VAI_Slide_2AEs_Soft_then_Stack.ipynb
VAI_Slide_2AEs_then_Stack_Soft.ipynb

```
original_dim = 784
In_layer = Input(shape=(original_dim,))
# encodede layer
encoding_dim = 30
E_layer = Dense(encoding_dim, activation='relu')(In_layer)
# decode layer
D_layer = Dense(original_dim, activation='sigmoid')(E_layer)

# put together
autoencoder = Model(In_layer, D_layer)

# loss='mean_squared_error'
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
autoencoder.fit(x_train, x_train, epochs=50, batch_size=256,
                validation_data=(x_test, x_test))
```

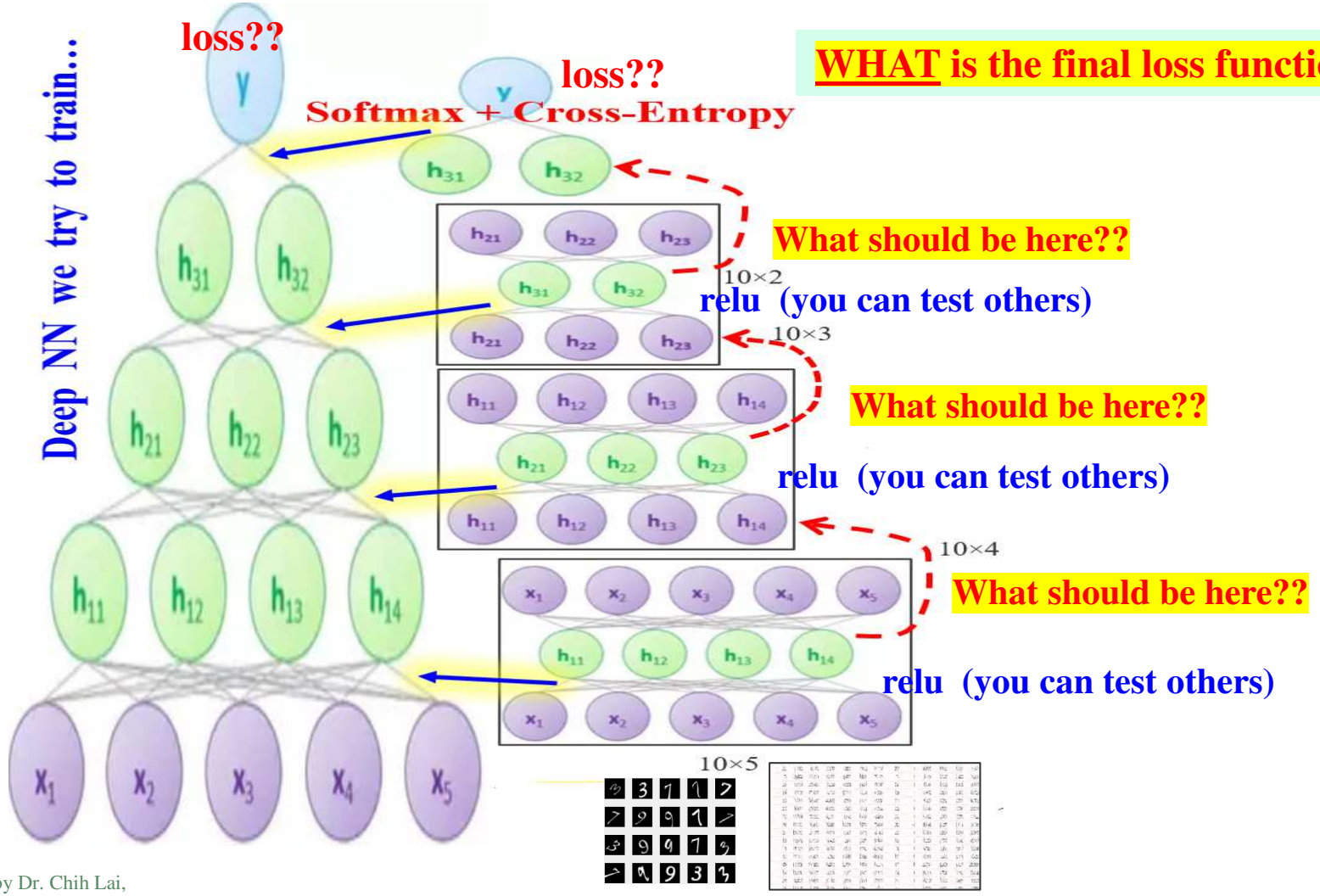
```
### Encoder
encoder = Model(In_layer, E_layer)
Z = encoder.predict(X)      # Z = 30-D
```

```
### Stacking w/ next AE
In_Z_layer = Input(shape=(encoding_dim,))
# get layer 1 of another AE
E_layer_Last = LastAE.layers[1]
# put together to create another AE
AnotherAE = Model(In_Z_layer,
                  E_layer_Another(…))

Next_Z = AnotherAE.predict(Z)
```

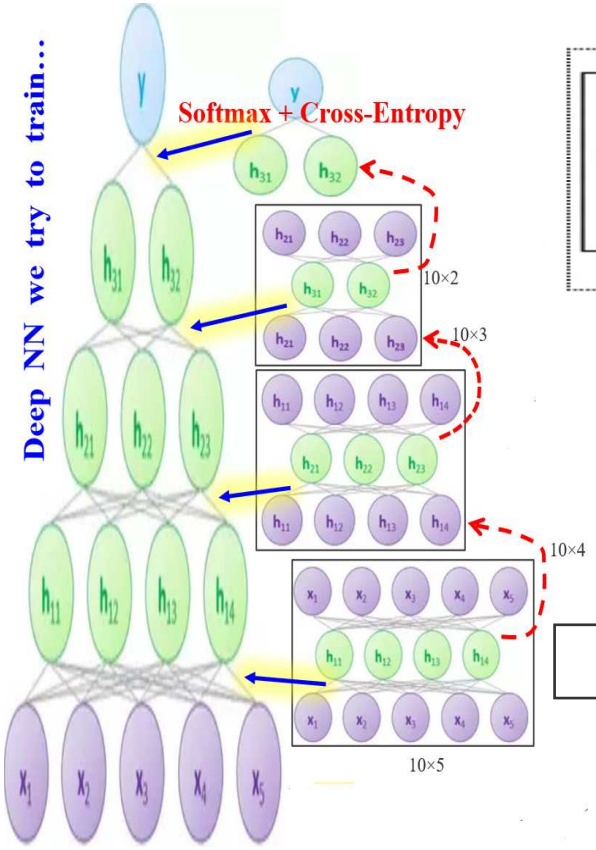
Choosing *Activation* & *Loss*!! for HW#4

- What should be the **activation function** at the **output layer of each AE??**
  - **Depends…… on???**
- Depend on input… **For *images*, use??** **For *numeric* records (z-score), use??**

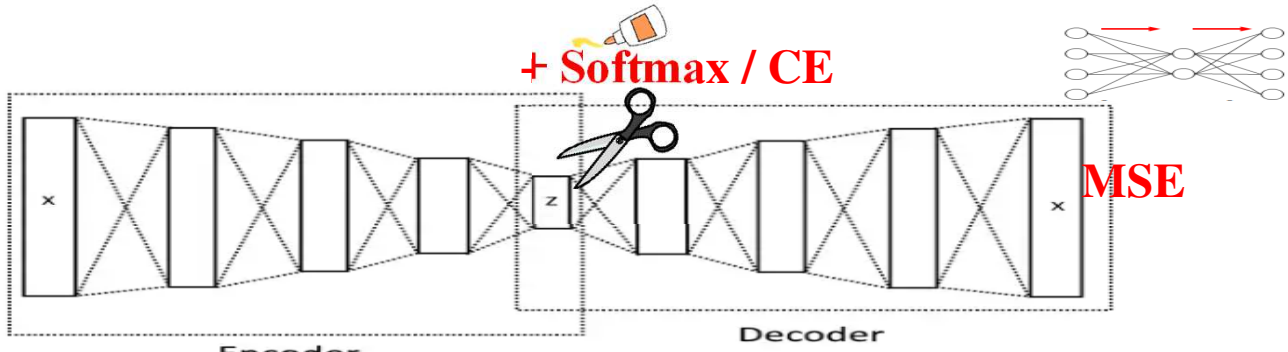


Why Not Just Train A Deep **End-to-End** AE? **Programming???**

- Why bother to do ***layer-wise*** pre-training w/ a classification head (softmax layer)?
- Just take 10 records w/ ***Y*** to train a **deep** AE, take encoder part, add a softmax layer?
  - Too many θs & not enough data (say dataset has only 20 records, although they **ALL** have ***Y***s)

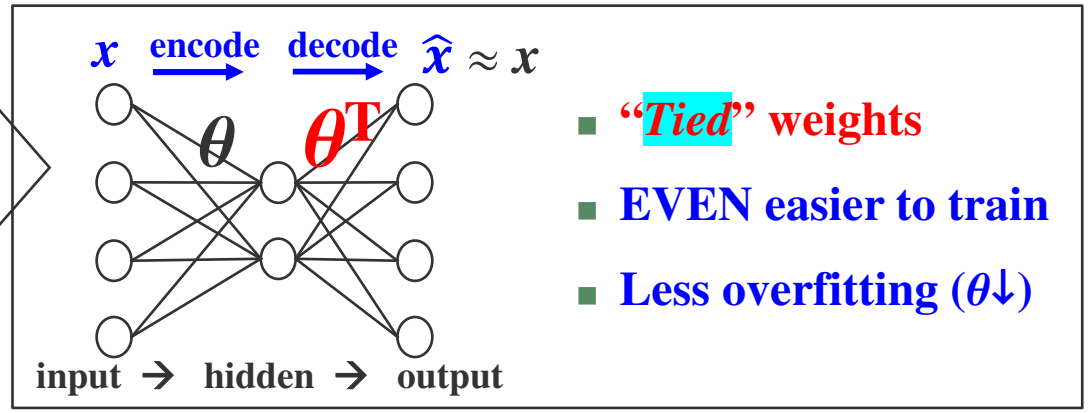


Layer-Wise AE



Just use random initial θs to train a *deep* End-to-End AE??

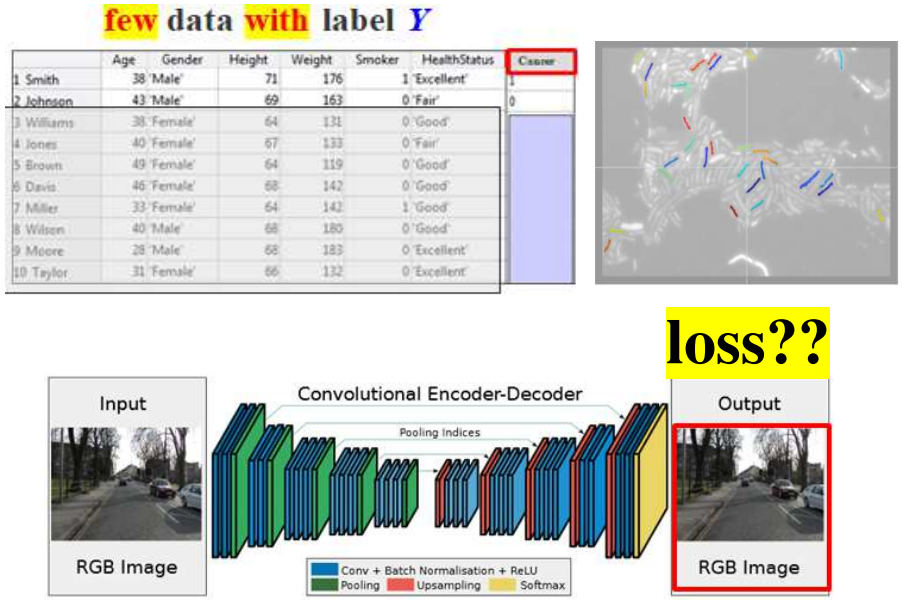
Not enough data!!



- **"*Tied*" weights**
- **EVEN easier to train**
- **Less overfitting ($\theta\downarrow$)**

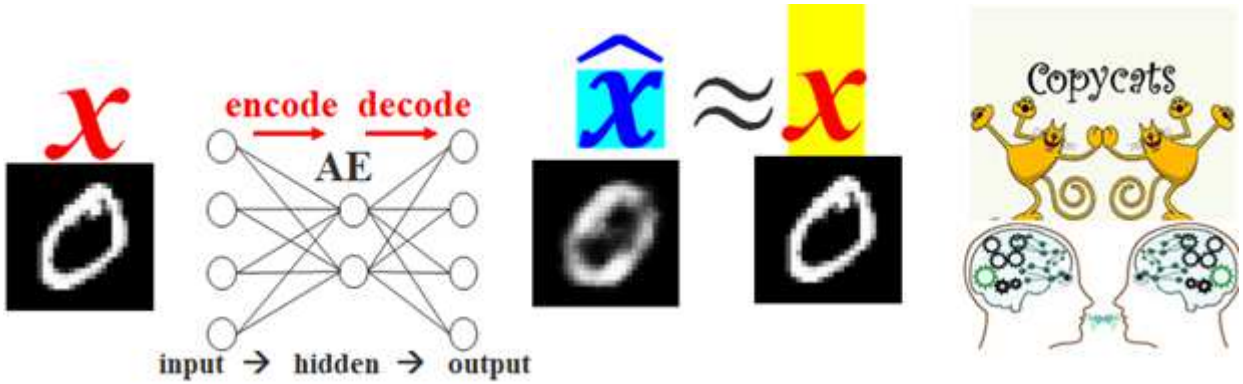
Outline

- *Semi*-Supervised Learning with Autoencoder (AE)
- Training / Fine-Tuning Autoencoder / Programming
  - Not Enough ***Y***s, Dimension Reduction ***Z***, Train Deep NN w/ Few ***Y***s, **MORE……**
- Homework 4 Review
- AE with Convolution, Denoising Autoencoder
- ***Variational Autoencoder***



MORE AE Applications

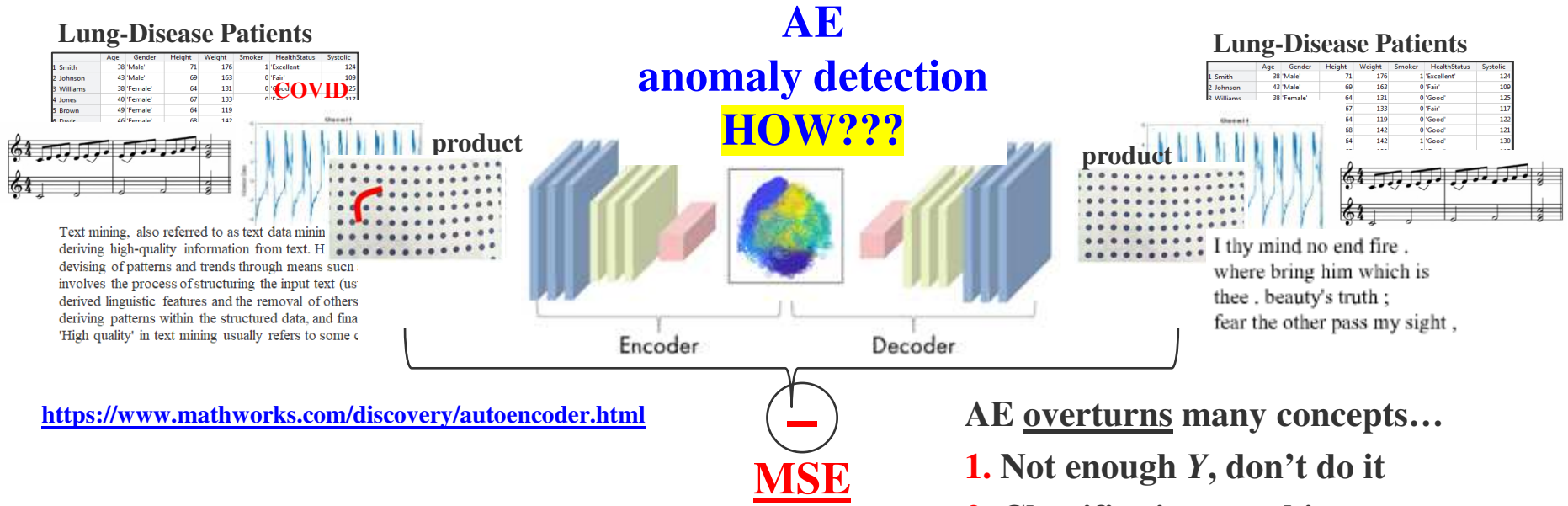
- Use AE to extract ***Z***-code from datasets that have **NO** ***Y*** (or few ***Y*****s**)
  - Serve as a ***dimensionality reduction*** tool & to extract hidden features
- Use AE to train **deep** NN for classifying datasets **with few *Y*s**
  - **Even** you have **enough *Y*s**, using AE to make training **very deep** NN easier
- **Anything else??**
- **Unbalanced** dataset?? **App 4**
  - Usually use **SMOTE**, or **kNN**, **SVM One-Class** classification, NB prior probability
  - **Use AE to generate more similar data… $\hat{x}$** … set ***X*** as ***Y*** for generating $\hat{x}$
- **and few more apps…**



More AE Applications & Extensions **App 5**

- AE can be trained **w/o** labels… so they are / can be **unsupervised (self-supervised)**
  - for numeric records, text/image generation, train deep NN… using dense, conv., or LSTM

- More applications: **anomaly detection**, **HOW??** **……denoising**



https://www.mathworks.com/discovery/autoencoder.html

MSE

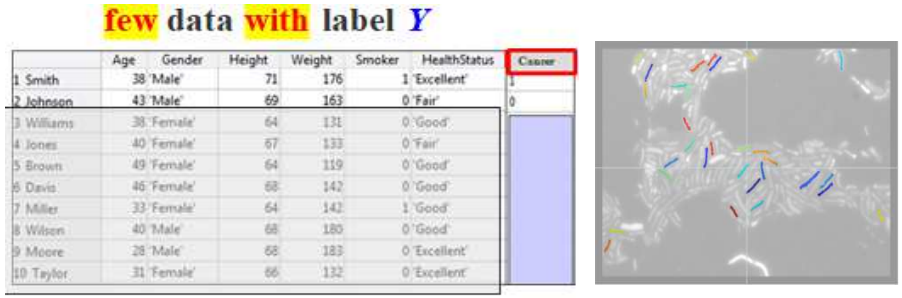
AE overturns many concepts…

1. **Not enough *Y*, don't do it**
2. **Classifier is everything**
3. **Overfitting is our *worst* friend**
4. **SMOTE is the only way**

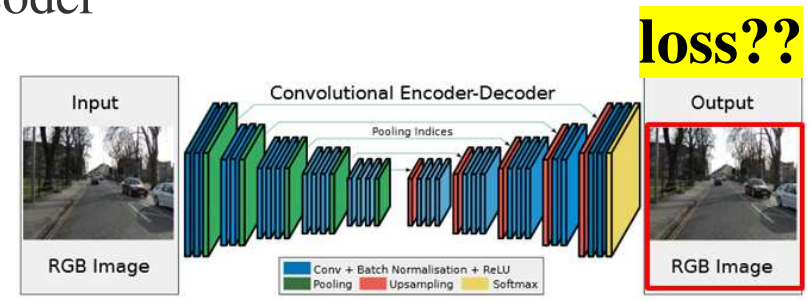
- Many variations of **AE** built for different tasks, i.e. ***Variational AE***, ***GAN***

Outline

- *Semi*-Supervised Learning with Autoencoder (AE)
- Training / Fine-Tuning Autoencoder / Programming
  - Not Enough ***Y***s, Dimension Reduction ***Z***, Train Deep NN w/ Few ***Y***s, MORE……
- **Homework 4 Review**

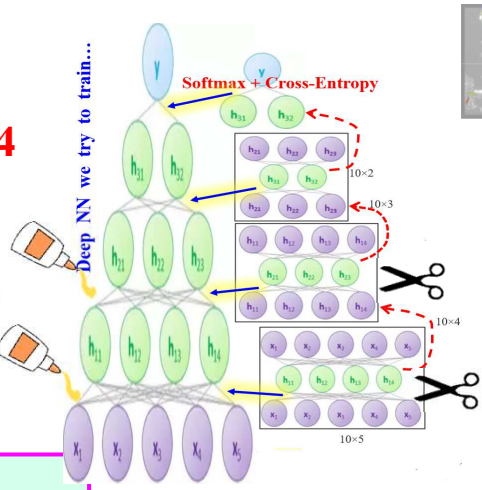


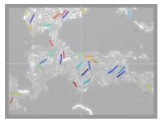
- AE with Convolution, Denoising Autoencoder
- Variational Autoencoder

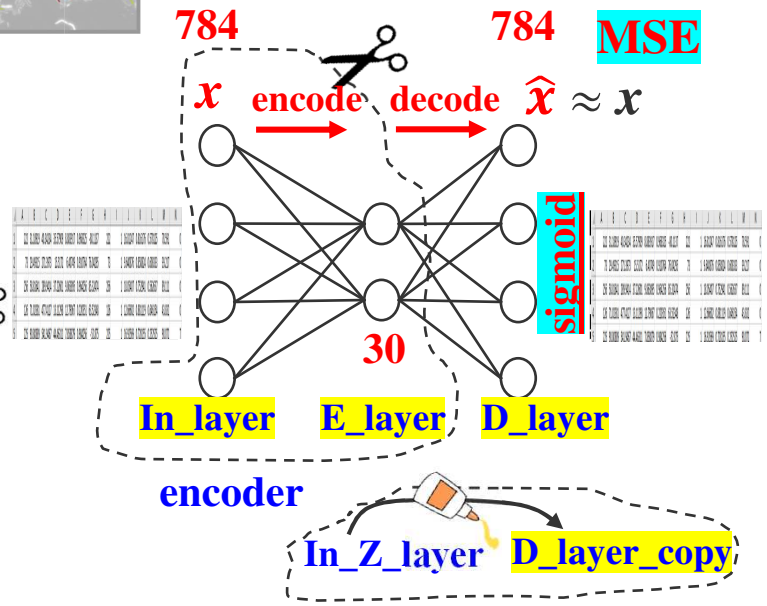


Keras Autoencoder Functional API for HW#4

- Build few AEs of **1 *layer***
  - https://wiseodd.github.io/techblog/2016/12/03/autoencoders/
  - https://blog.keras.io/building-autoencoders-in-keras.html







VAI_Slide_AE_1_layer.ipyng
VAI_Slide_2AEs_Soft_then_Stack.ipynb
VAI_Slide_2AEs_then_Stack_Soft.ipynb

```
original_dim = 784
In_layer = Input(shape=(original_dim,))
# encodede layer
encoding_dim = 30
E_layer = Dense(encoding_dim, activation='relu')(In_layer)
# decode layer
D_layer = Dense(original_dim, activation='sigmoid')(E_layer)

# put together
autoencoder = Model(In_layer, D_layer)

# loss='mean_squared_error'
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
autoencoder.fit(x_train, x_train, epochs=50, batch_size=256,
                validation_data=(x_test, x_test))
```

```
### Encoder
encoder = Model(In_layer, E_layer)
Z = encoder.predict(X)      # Z = 30-D
```

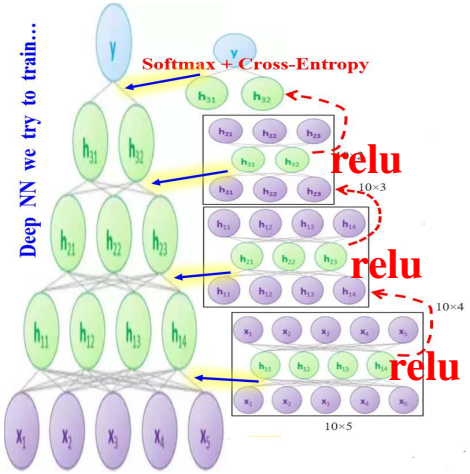
```
### Stacking w/ next AE
In_Z_layer = Input(shape=(encoding_dim,))
# get layer 1 of another AE
E_layer_Last = LastAE.layers[1]
# put together to create another AE
AnotherAE = Model(In_Z_layer,
                  E_layer_Another(…))

Next_Z = AnotherAE.predict(Z)
```

Compare AE to NN Using Small Data (**relu**)

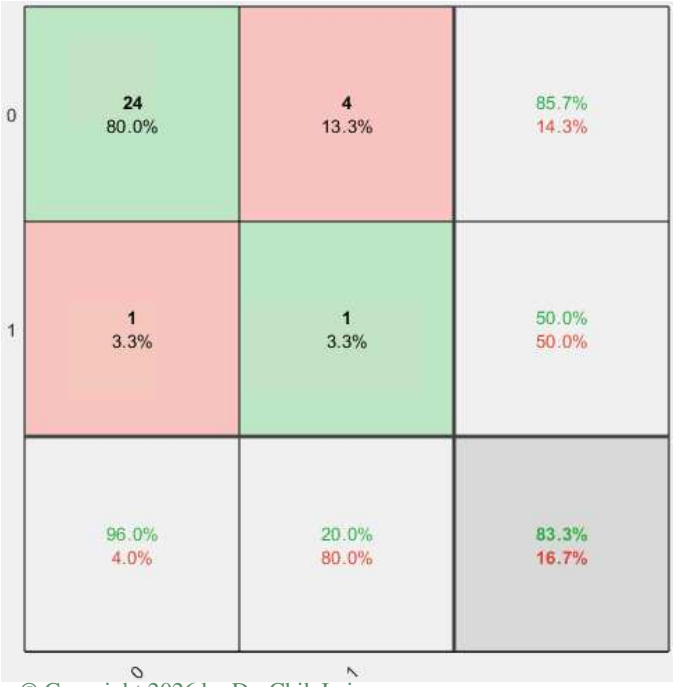
- **AE** on cell DNA data (13-D), not difficult data
  - But use **only** 30 records from 1217 (**2.5%**)
  - **4** networks = **3 AEs** (size **10**, **7**, **4**) + **softnet** (2)
  - Train **50** epochs for each AE
  - *Middle* **relu** activation & initial L.R. = 10^{-3}

```
softnet_layers = [
  featureInputLayer(4)
  fullyConnectedLayer(2)
  softmax activation
  B.C.E. ];
```

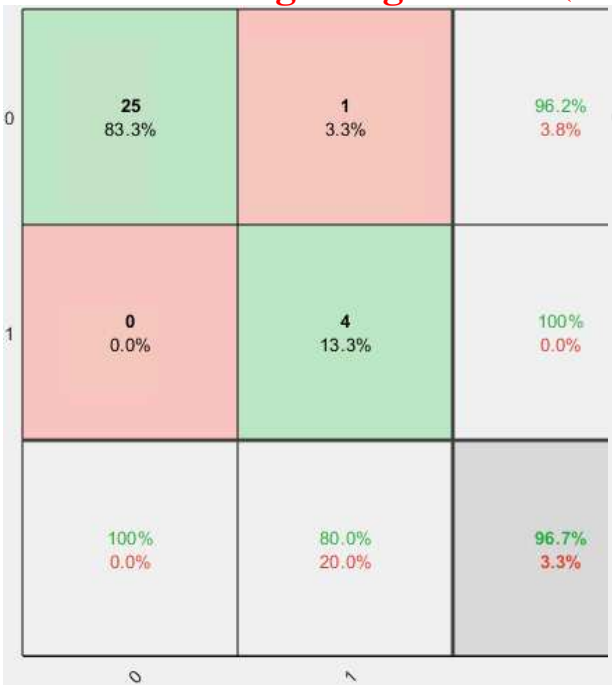


- Regular **NN** (size 10➔7➔4➔2) w/ **same** **30 rows** for 200 (50×4) or less epochs w/ **relu** & L.R. = 10^{-3}

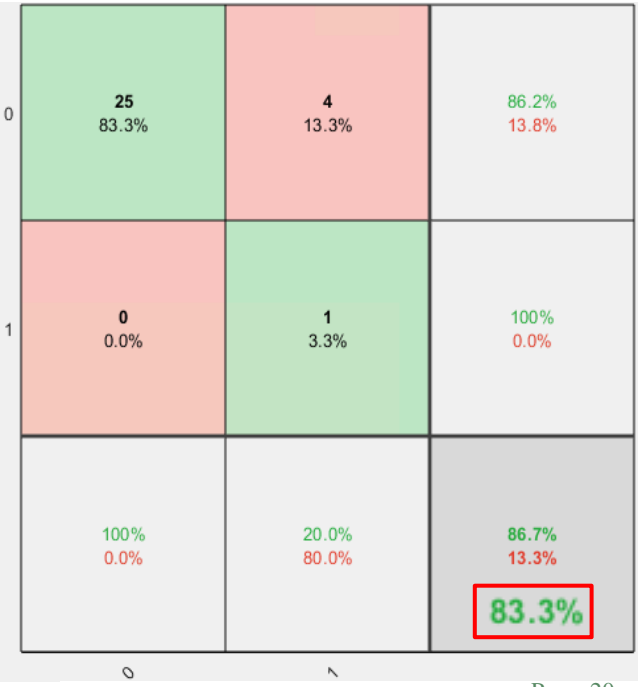
no fine tuning



AE fine tuning = regular NN (200)

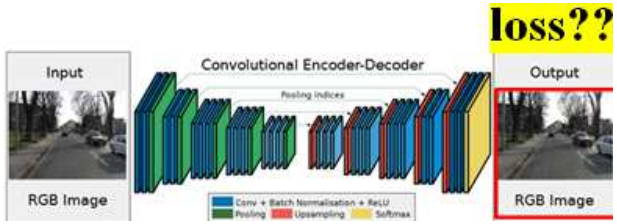


regular NN, 50 or 100 epochs



Outline

- *Semi*-Supervised Learning with Autoencoder (AE)
- Training / Fine-Tuning Autoencoder / Programming
  - Not Enough ***Y***s, Dimension Reduction ***Z***, Train Deep NN w/ Few ***Y***s, MORE……
- Homework 4 Review
- **AE with Convolution, or with U-Net**
- **Denoising Autoencoder**
- Variational Autoencoder



myAE.fit(*x_Img*, *x_Img*, …)

Original **gray**

U-Net

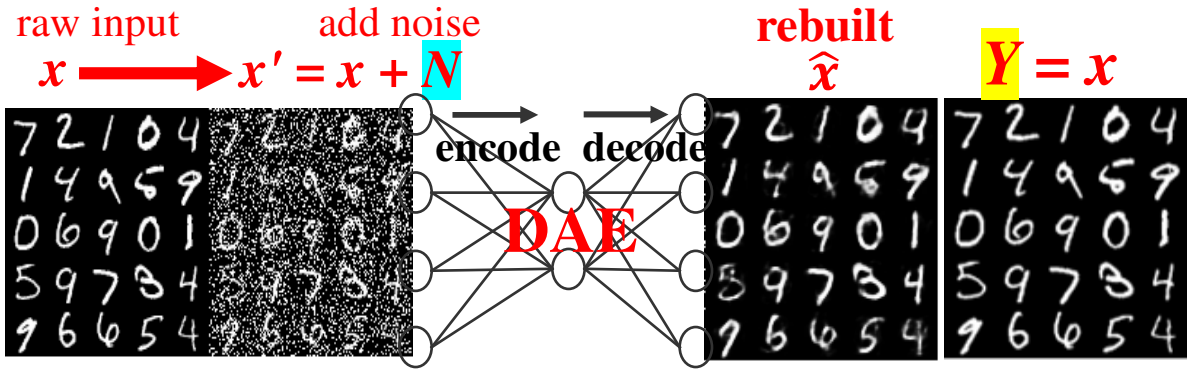
Reconstructed

VAI_Slide_AE_UNet_RebuildDog.ipynb

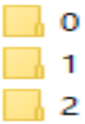
```
## last layer must use "sigmoid" for AE rebuild function
outputs=Conv2D(input_shape[-1], (3, 3), activation='sigmoid')(d4)
outputs = Resizing(256, 256, interpolation='bilinear')(outputs)
```

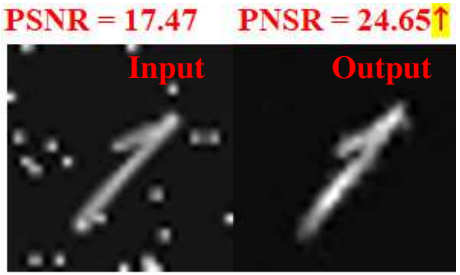
App 6
Denoise AE (**DAE**) for Denoising (**Image or Numeric…**)

- Add noise to X for AE to learn **even better** latent Z!!!
  - Without noise, AE simply learn copying input
  - Noise as **regularizer** for better generalization
  - Force learning more robust patterns w/ different noise



- Use total 30 images of digits 0, 1, 2
  - Train 27 training for 300 epochs & 3 testing
  - ***P**eak **S**ignal-to-**N**oise **R**atio* (**larger better**)

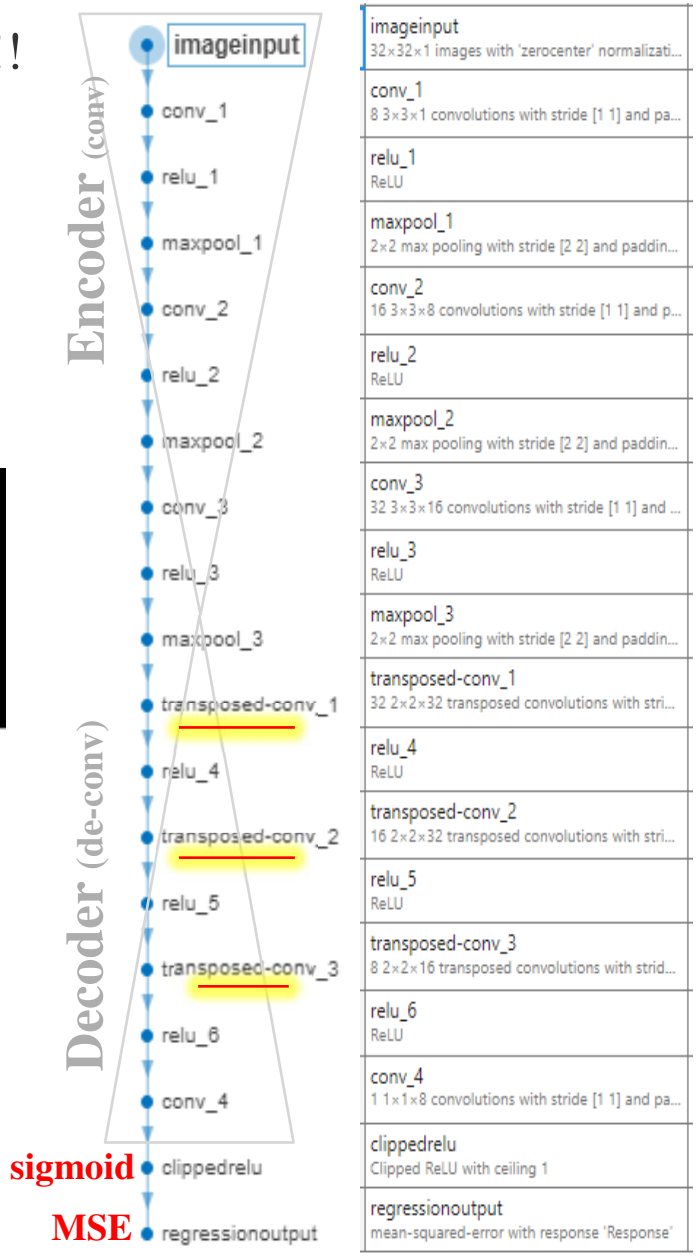




VAI_Slide_AE_CNN_Config.ipynb

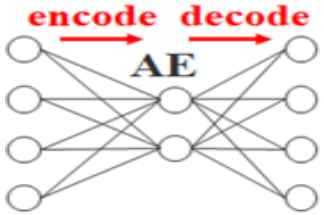
VAI_Slide_AE_Noisy_Digits.m

Data_DigitImg_012_Each10_InSubFolders.zip



Outline

- *Semi*-Supervised Learning with Autoencoder (AE)
- Training / Fine-Tuning Autoencoder / Programming
  - Not Enough ***Y***s, Dimension Reduction ***Z***, Train Deep NN w/ Few ***Y***s, MORE……
- Homework 4 Review
- AE with Convolution, Denoising Autoencoder
- ***Variational Autoencoder***



Using Dense

VAI_Slide_AE_1_layer.ipyng
VAI_Slide_2AEs_Soft_then_Stack.ipynb
VAI_Slide_2AEs_then_Stack_Soft.ipynb

VAI_Slide_AE_Stacking.m
VAI_Slide_AE_Stack_NumData.m

Using Convolution

VAI_Slide_AE_CNN_Config.ipynb
VAI_Slide_AE_UNet_RebuildDog.ipynb

VAI_Slide_AE_Noisy_Digits.m